Lab 0 - Programming under Ubuntu Linux

In this lab, you will learn some basic knowledge of WSL2 and C/C++, as well as how to program under the Ubuntu Linux environment instead of the Microsoft Windows environment.

0.1 Introduction of WSL2

WSL2 (Windows Subsystem for Linux 2) is a compatibility layer developed by Microsoft that allows users to run Linux distributions natively on Windows 10 and 11 systems. WSL2 improves upon the original WSL (WSL1) by using a real Linux kernel running in a lightweight virtual machine (VM), providing better performance, compatibility, and features.

Here are some key points about WSL2:

1) Real Linux Kernel: WSL2 runs a full Linux kernel inside a virtualized environment, which allows for full system call compatibility. This means it can run more complex applications, such as Docker, which were not fully supported on WSL1.
2) Performance: WSL2 offers significantly better performance than WSL1, especially for tasks involving heavy file system operations and I/O. This is due to the use of the Linux kernel and improved file handling mechanisms.
3) File System Access: You can easily access Linux files from Windows via the \\wsl$\ path, and you can also access your Windows file system from the Linux environment.
4) Lightweight Virtual Machine: Unlike traditional VMs, WSL2 uses a lightweight VM, so it has a lower resource overhead, faster boot times, and better integration with Windows.
5) Integration with Windows: WSL2 integrates well with Windows, allowing you to run Linux tools alongside Windows applications, share network configurations, and copy files between systems.
6) Compatibility with Docker: WSL2 supports Docker natively, making it possible to use Docker containers on Windows without the need for a full VM, which was a limitation with WSL1.

Note: To use WSL2, you will need to enable the WSL feature in Windows, install a Linux distribution from the Microsoft Store, and optionally switch the distribution to use WSL2 if it defaults to WSL1.

0.2 Installation of WSL2

Pre-conditions

- Windows 10 version 2004 and later or Windows 11
- 64-bit systems architecture

Step 1: Enable WSL

1) Open PowerShell as an administrator:
   Right-click on the Start menu and select Windows PowerShell Admin) or Command Prompt Admin).
2) Run the following command to enable the Windows Subsystem for Linux:
   dism.exe /online /enable-feature /featurename:Microsoft-Windows- Subsystem-Linux /all /norestart

3) This command enables the WSL feature, but a restart is recommended after all steps are completed.

Step 2: Enable the Virtual Machine Platform

1) Run the following command in PowerShell Administrator mode) to enable the Virtual Machine Platform, which is required for WSL 2
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
2) Restart the computer after this step to ensure the changes take effect.

Step 3: Download and install the WSL 2 Linux Kernel Update Package

1) Automatic Update Method: Open PowerShell in administrator mode and run:
wsl –update
2) Manual Update Method (if the automatic update fails):
Download the latest WSL 2 Linux kernel update package from the official Microsoft website.
https://wslstorestorage.blob.core.windows.net/wslblob/wsl_update_x64.msi
Run the downloaded *.msi* file to manually install the Linux kernel.

Step 4: Set WSL 2 as the Default Version

1) Open PowerShell.
2) Run the following command to set WSL 2 as the default version for new Linux installations:
wsl --set-default-version 2

Step 5: Install the Linux Distribution

Option 1: Using the Microsoft Store

- Open the **Microsoft Store**.
- Search for **Ubuntu 22.04 LTS**.
- Click **Get** or **Install** to download and install Ubuntu.

Option 2: Using PowerShell

- Run the following command in PowerShell to install Ubuntu 22.04 directly: Xi'An Jiao Tong University AI and Intelligent Robots
wsl --install -d Ubuntu-22.04
- This method will automatically download and install Ubuntu 22.04 LTS.

Step 6: Initialize the Newly Installed Linux Distribution

1) Start the newly installed Linux distribution from the Start menu or by typing Ubuntu 22.04 in PowerShell or Command Prompt.
2) Wait for the installation to complete. This may take a few minutes.
3) Create a username and password for your Linux environment. These credentials are for the Linux distribution only and do not need to match your Windows credentials.

Step 7: Verify the WSL Installation

1) Open PowerShell.

2) Run the following command to list all installed Linux distributions and their WSL versions:
   wsl --list --verbose
   The output should show your installed distribution (e.g., Ubuntu-22.04 and the version set to WSL2.

Optional Configurations

1) Change the Default Linux Distribution:
   If you have multiple distributions installed, you can set the default one by running:
   wsl --set-default
2) Access Linux Files from Windows:
   You can access your Linux file system via \wsl$\Ubuntu-22.04\ in File Explorer.

Updating WSL:

You can always update to the latest WSL version with: *wsl --update*

Fault Resolution

1) **Ensure Windows is Up to Date**:
   Make sure your Windows installation is fully updated by checking for updates in **Settings** > **Update & Security** > **Windows Update**.
2) **Check Virtualization in BIOS**:
   Verify that virtualization is enabled in your computer's BIOS/UEFI settings.
3) **Verify Hyper-V is Enabled**:
   Some users may need to enable Hyper-V manually. You can enable it using the following PowerShell command:
   dism.exe /online /enable-feature /featurename:Microsoft-Hyper-V-All /all /norestart
4) **Reset the WSL Configuration**:
   - If you experience problems with WSL, you can reset the configuration with: *wsl --unregister <DistributeionName>*
   - This will remove the specified Linux distribution and allow for a fresh reinstallation.

0.3 Run a C/C++ program on Ubuntu Linux terminal

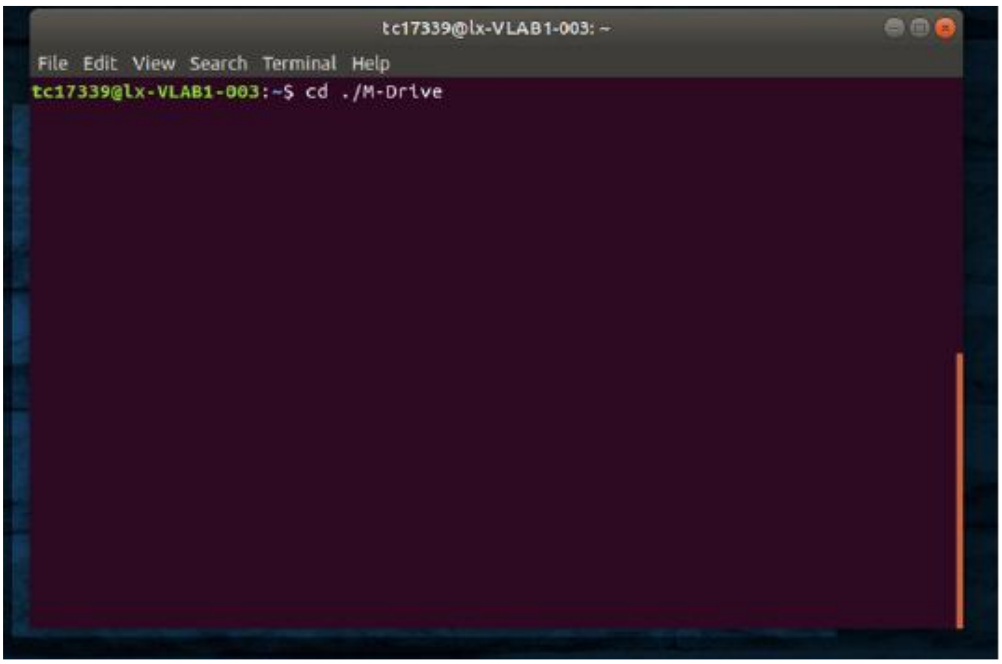
Step 1: Use your username and password to login the Ubuntu Linux system.

Step 2: Using (Ctrl + Alt + t) to open a terminal.

Step 3: Input the following command to enter your M-Drive

cd ./M-Drive

You can see the figure below.



Step 4 Type the following command to check your ***gcc*** version.

Note that ***gcc*** or g++ complier has been installed for you to use.

gcc –v or *g++ –v*

Step 5 Go to the folder ***Documents*** by typing the following commands:

```
cd Documents
mkdir programs
cd programs
```

Step 6 Open a file using *gedit* editor.

```
gedit hello.cpp
```

Step 7 Copy the following code into the file:

```
#include<iostream>
using namespace std;
int main()
{
  cout <<"\nHello World, Welcome to my 1st program on Ubuntu Linux\n" << endl;
  return 0;
}
```

Save the file and exit.

Step 8 Compile and run the program using the following commands:

```
g++ hello.cpp
./a.out
```

The following message will show output on the terminal.

"Hello World,

Welcome to my 1st C++ program on Ubuntu Linux"

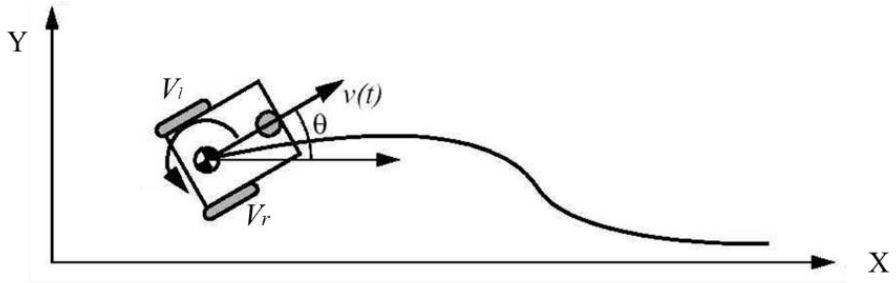
Tips and Tricks for Using Linux Command Line

- You can use the **clear** command to clear the terminal in Linux.
- **TAB** can be used to fill up in terminal. For example, you just need to type "**cd Doc**" and then **TAB** and the terminal fills the rest up and makes it "**cd Documents**".
- **Ctrl+C** can be used to stop any command in terminal safely. If it does not stop with that, then **Ctrl+Z** can be used to force stop it.
- You can exit from the terminal by using the **exit** command.
- You can power off or reboot the computer by using **sudo halt** and **sudo reboot**.
- For more info:*https://maker.pro/linux/tutorial/basic-linux-commands-for-beginners*

0.2 Work on Task A

All software you need in CE801 assignments have been installed in the Lab computers, such as Ubuntu 22.04 and ROS2.

Task A-1: The following figure describes a differentially driven robot in a 2D space. The robot position is (*x, y*) and its heading is *θ*.



The discrete form of its kinematic equations is shown below:

$$x(k+1) = x(k) + \frac{V_r + V_L}{2}\cos(\theta(k)) * \Delta t$$

$$y(k+1) = y(k) + \frac{V_r + V_L}{2}\sin(\theta(k)) * \Delta t$$

$$\theta(k+1) = \theta(k) + \frac{V_r - V_L}{W} * \Delta t$$

where W is the wheelbase of the robot, i.e., the distance between two rear wheels.

V_l and V_r are the velocities of the left and right wheels, respectively.

Δt is the cycle time.

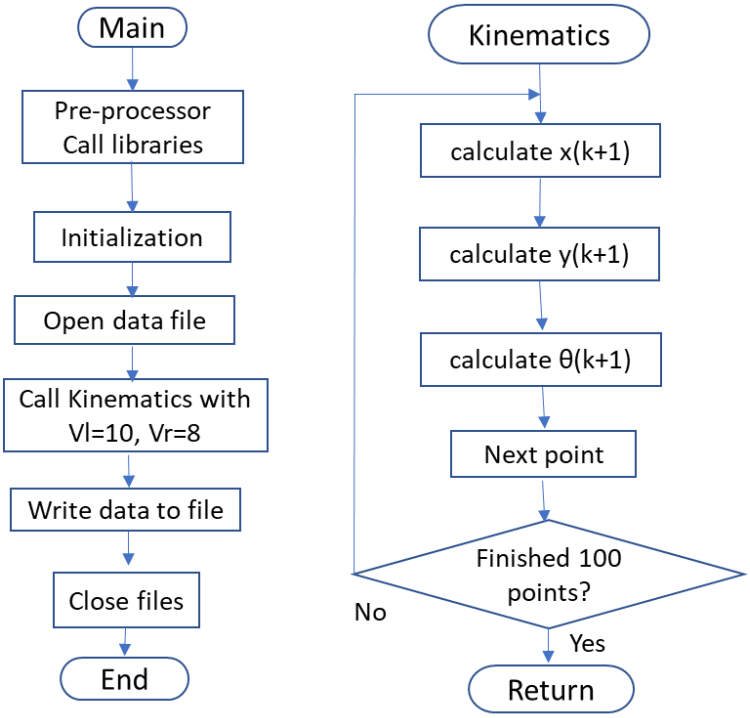
Suppose you have been given the following parameters:

$V_l = 10cm/s;$ $V_r = 8cm/s;$ $W = 30\ cm;$ $\Delta t = 1;$

$x(0) = 30cm;$ $y(0) = 30cm;$ $\theta(0) = \pi/4$

Your task is to write C/C++ code to generate the robot trajectory (x_k, y_k) when k = 0,1,2,3,4,5,6,7…100. All the trajectory points should be saved into a file for plotting, which is also known as odometry data or dead-reckoning data.

Now let us follow the following flowchart to do the coding. However, you may write it in your way.



The following steps aim to help you do the programming and graph plotting.

Step 1 Open terminal.

Step 2 Type the commands to go to the folder ***Documents/programs***

$ cd Documents/programs

Step 3 Open the file using any editor

$ gedit odometry.cpp

and add the parameters given above to the code.

```
#include<math.h>
#include<iostream>
#include<fstream>

#define   PI   3.14159265
int wheelbase=30, dt=1;
const int SIZE=100;
double vl=10, vr=8;
double rob_x[SIZE]={30}, rob_y[SIZE]={30}, rob_theta[SIZE]={PI/4};

// To generate the robot trajectory using robot kinematic equations
int robot_kinematics(double left_vel, double right_vel)
{
    int i;
    for(i=0; i<SIZE; i++)
    {
        // Put your code for implementing kinematic equations here
    }
```

```
    return 0;
}
// You should write main function into the file.
FILE *fp;    // data file for saving the trajectory data
int main(int argc, char **argv)
{
    int i;
    fp = fopen ("trajectory1","w");
    robot_kinematics(vl, vr);
    fprintf(fp, "%d, %d  \n", 30, 30);
    for(i=1; i<SIZE; i++){
        fprintf(fp, "%f, %f  \n", rob_x[i], rob_y[i]);
    }
    fclose(fp);
    return 0;
}
```

Step 4 Save the file and exit.

Step 5 Type the following command to compile your odometry calculation code.

$ g++ odometry.cpp

Step 6 Run your code.

$ ./a.out

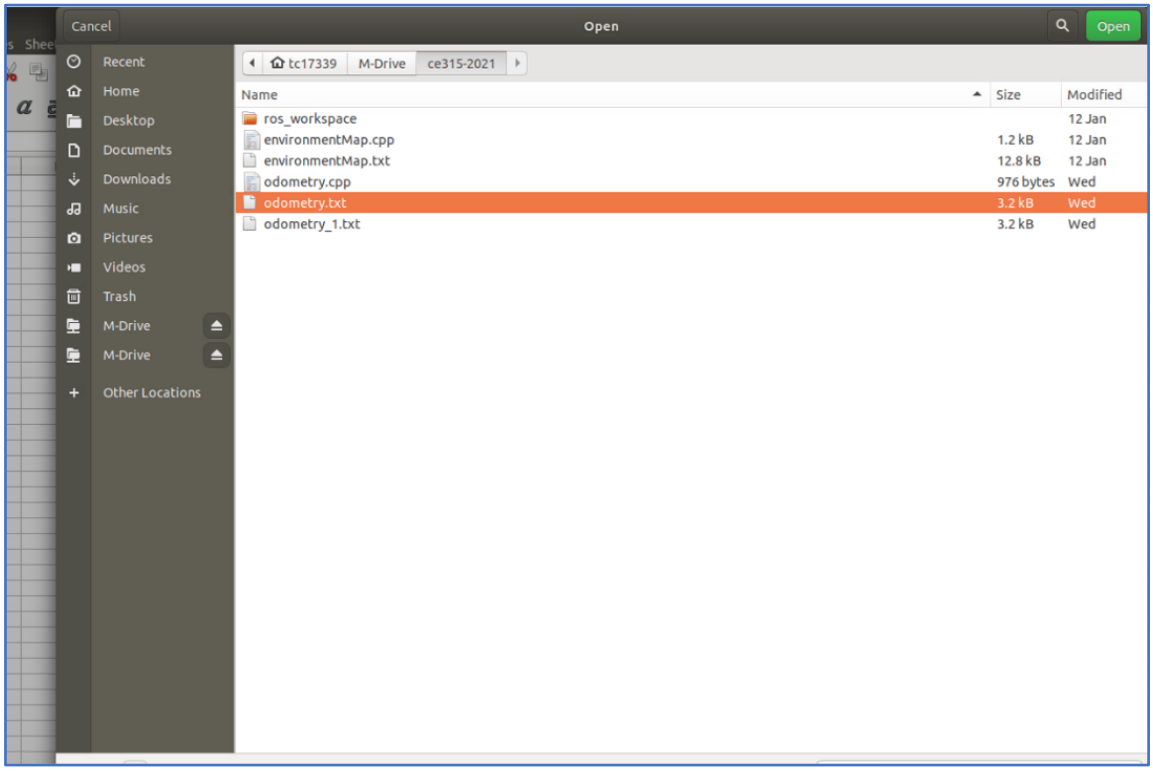
At this point, you should have a data file *trajectory1* file being generated in your M_Drive. Your next task is to plot it using LibreOffice Calc in Ubuntu applications.

Plot your results using LibreOffice Calc in Ubuntu applications

Step 1 Search and select LibreOffice Cals from Activities in Ubuntu desktop.

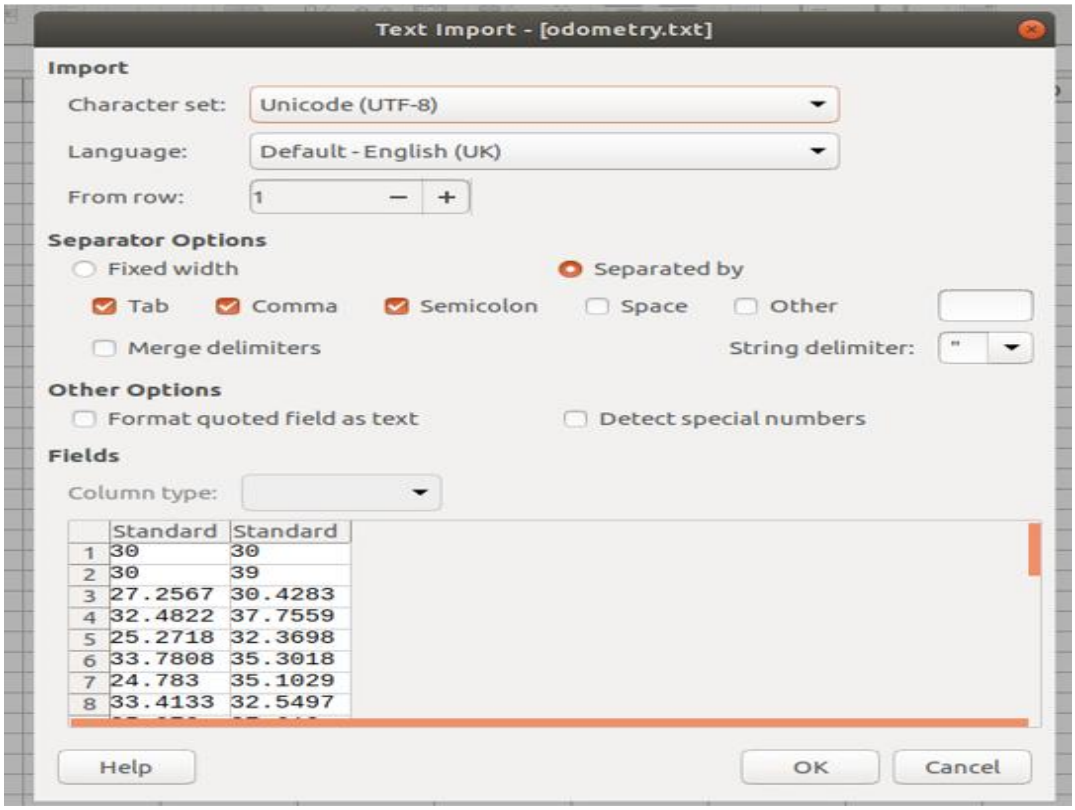
Step 2

Once you open the LibreOffice Cals, you should open your data file (e.g., odometry.txt in this case) from **File → Open.** In the pop-up window, go to your file location (e.g., ~/M-Drive/ce315-2021/odometry.txt in this case). You can select the file and then select **Open.**



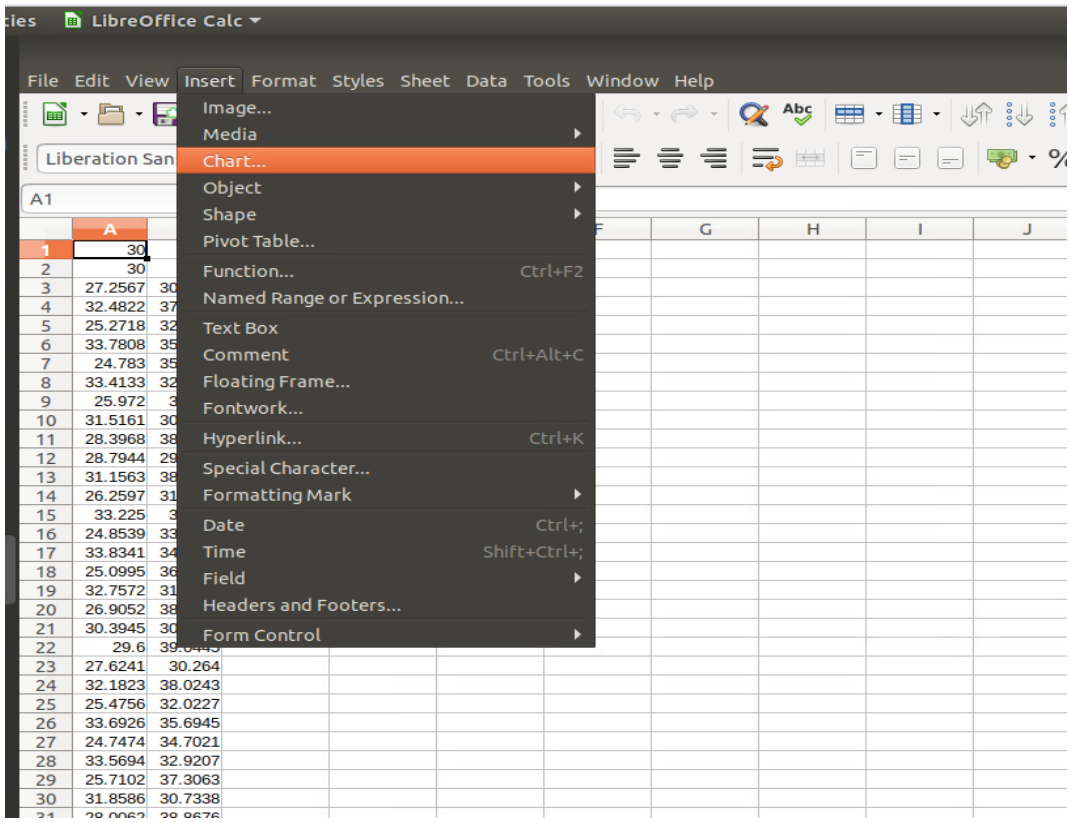
Step 3

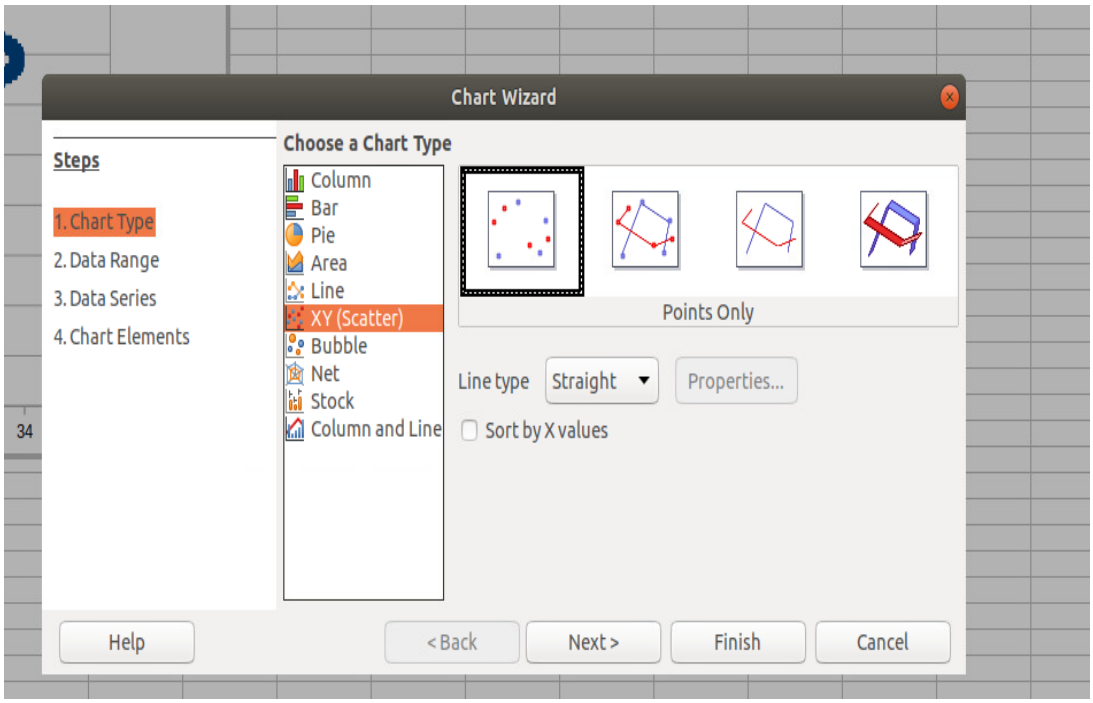
In the pop-up window, you may select one of Separators for your data (Tab, Comma, Semicolon) based on the separator used in your data. Then select Ok as the figure shown below.



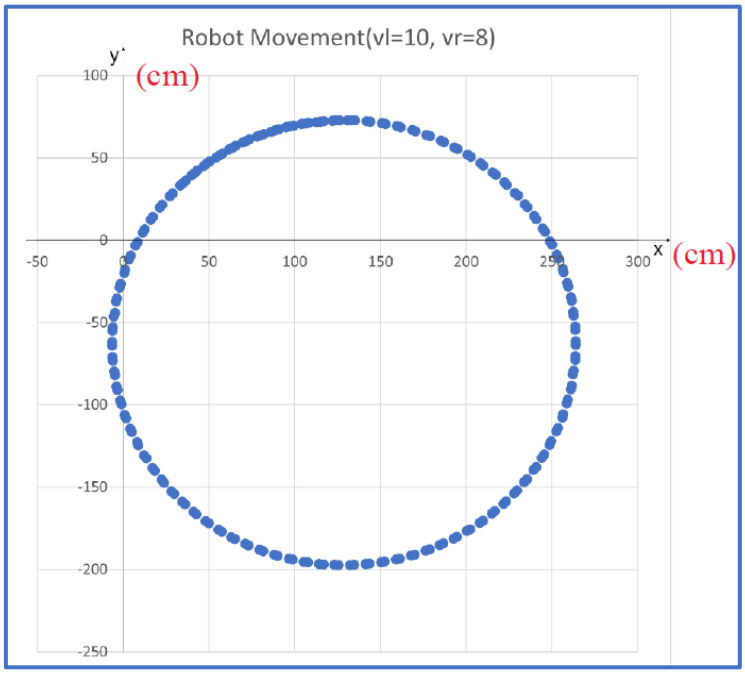
Step 4

Now you can go to Insert→Chart→Chart Type→XY (Scatter), choose left diagram (Points Only) and select Finish.





Then, you can view the shape of your data below.



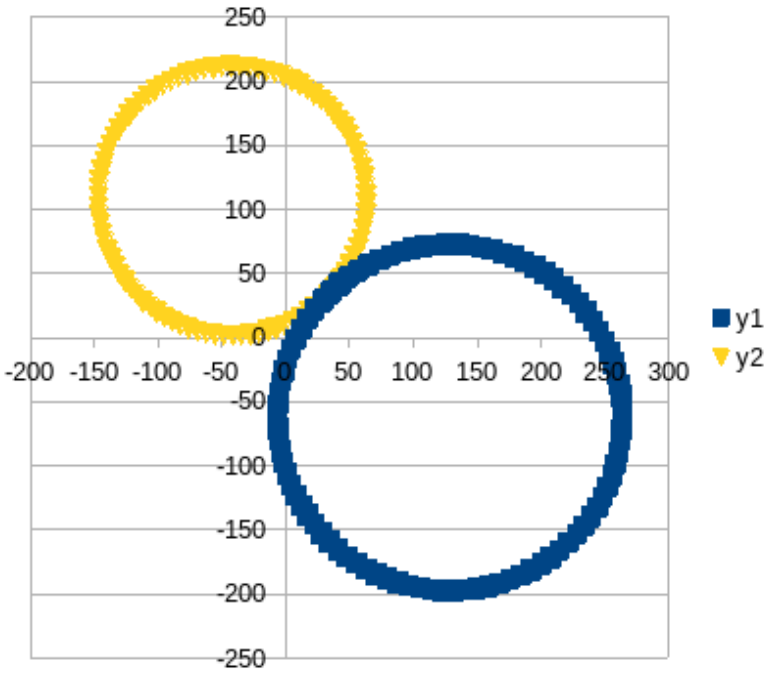
Task A-2: Improve your C/C++ programming skill.

Suppose you have been given the following new parameters:

$$V_l = 5cm/s; \qquad V_r = 7cm/s; \qquad W = 30\ cm; \qquad \Delta t = 1;$$

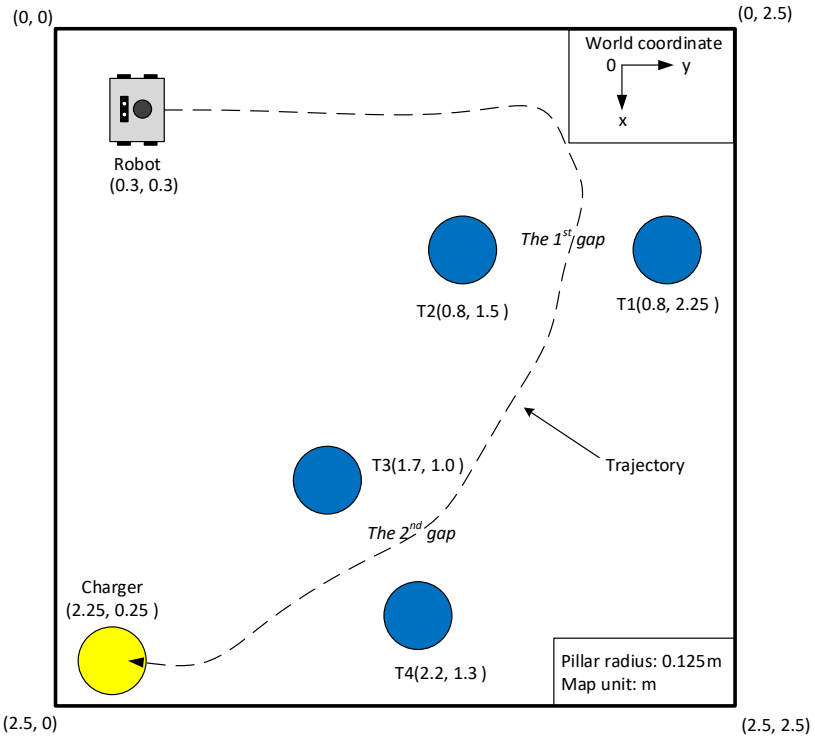
$$x(0) = 30cm; \qquad y(0) = 30cm; \qquad \theta(0) = \pi/4$$

You should use a module approach to revise your C/C++ code created in Task A-1 so that it can generate two robot trajectories (x_k, y_k) at two pairs of different velocities. Note k = 0,1,2,3,4,5,6,7…100, and all the trajectory points should be saved into a file for plotting.

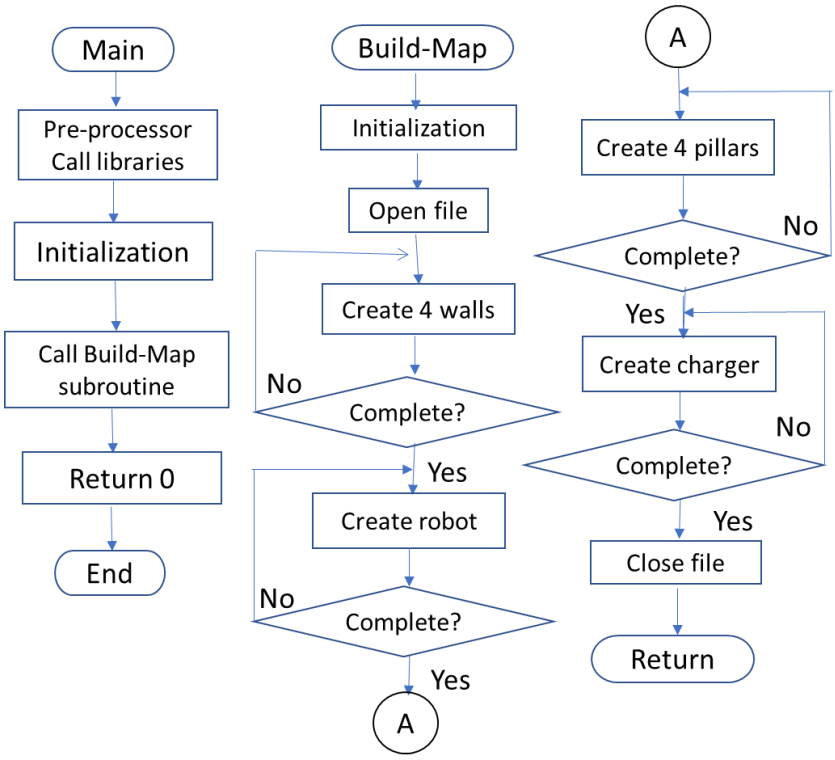


0.3 Work on Task B

In this task, you should write C/C++ code to create an environment map for its display in LibreOffice Calc in Ubuntu. The following figure shows the robot and its environment (4 pillars and 1 charge station in 1 square room) that is used for your assignment.



Based on it, you should write C/C++ code to generate this map and save the data into a file for its display in Excel. You may follow the following flowchart to write your code. However, you may write it in your way.



Step 1. To start your coding using the following commands:

cd M-Drive/Documents/programs

gedit EnvironmentMap.cpp

Step 2: After completing your coding, save the file. The commands below for your reference.

```
#include<iostream>
#include<fstream>
#include<math.h>
using namespace std;
#define PI 3.14159265

// To initialize all the parameters by using data in the figure above.
double x0[5]={0.8, 0.8, 1.7, 2.2, 2.25};
double y0[5]={1.5, 2.25, 1.0, 1.3, 0.25};
double r=0.125, wallLength=2.5;
double RD=PI /180;


// The following code is to create a charger based on its centre.
for (int i=0; i<360; i=i+20) {
    map<<x0[4]+r*cos(i*RD)<<' '<<y0[4]+r*sin(i*RD)<<endl;}

// For your reference, the following code is used to generate one wall.
for (double j=0 ; j<wallLength; j=j+0.01){
   map << j << ' ' << 0 << endl;}
```

Step 3: Type the following command to compile your odometry calculation code.

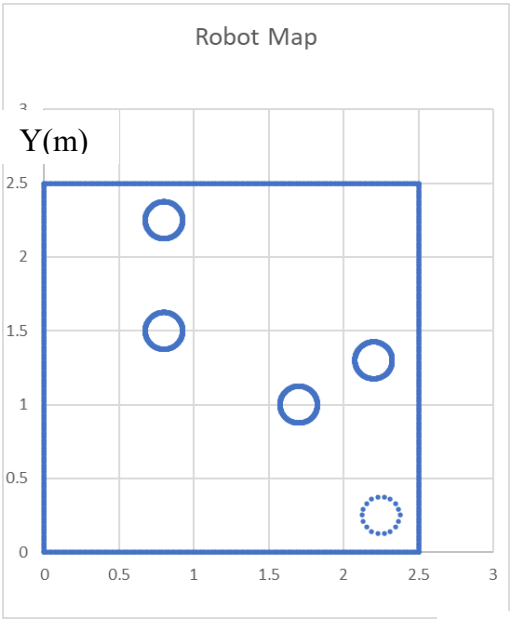
g++ EnvironmentMap.cpp

Step 4: Run your code.

./a.out

At this point, you will have the "EnvironmentMap.txt" file being generated in your M-driver. Next task is to plot it using *LibreOffice Calc* in Ubuntu applications.

Step 5: Plot *EnvironmentMap* using LibreOffice Calc in Ubuntu applications, which is shown at the right.



Step 6: To add a square robot (0.25m edge and centred at (0.3, 0.3) into the map, you should add your code by using the following command.

gedit EnvironmentMap.cpp

Step 7: Using the following commands to compile and run your revised code.

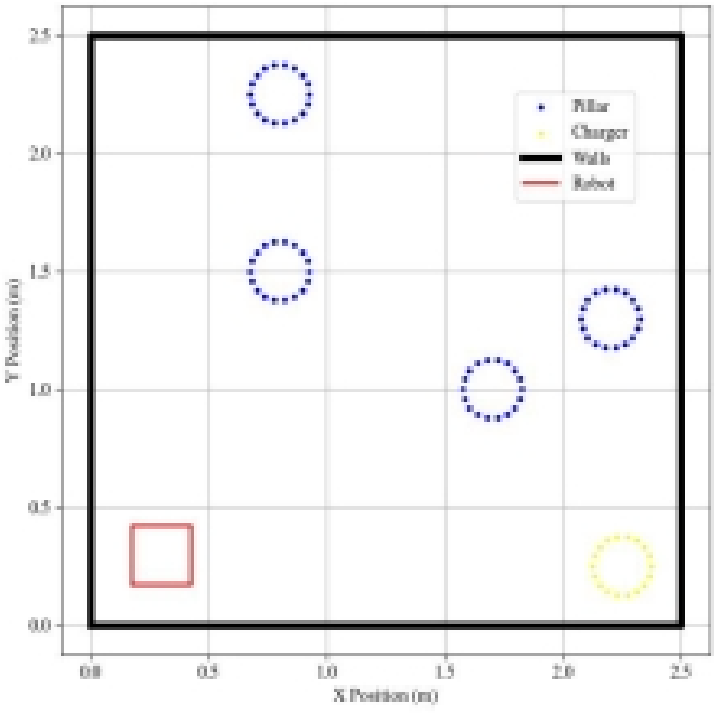
g++ EnvironmentMap.cpp

./a.out

At this point, a new "EnvironmentMap.txt" file is generated in your M-driver.

Step 8: Plot *EnvironmentMap* using LibreOffice Calc in Ubuntu applications.

You can see the graph below.



1.4 Merge Task A and Task B together

In this task, you should merge Taks A and Task B code to generate two robot trajectories and an environment map together. Also, you should learn how to create a header file and include it into main C file.

Step 1: create the main function below.

cd M-Drive/Documents/programs
gedit Trajectory_Map.cpp

```
#include "robot_function.h"     // Include the robot functions
int main(int argc, char **argv){
    // Initialize the variables
    int pillar_no = 4, charger = 4, P_D = 10, C_D = 30;
    double T1_vel[2] = {9, 7}, T2_vel[2] = {6, 8};
    double wall_o = 0, wallLength = 250, c_r = 12.5;
    double  rob_edge_start = 17.5, rob_edge_end = 42.5;
    double wall_step = 0.03, robot_step = 0.02;
    double x_c[5] = {80, 80, 170, 220, 225};   //x for centre of pillars and charger
    double y_c[5] = {150, 225, 100, 130, 25}; //y for centre of pillars and charger

    fp = fopen("Trajectories-Map","w"); //for storing two trajectories and one map

    robot_kinematics(T1_vel[0], T1_vel[1]); // to generate the 1st trajectory
    robot_kinematics(T2_vel[0], T2_vel[1]); // to generate the 2nd trajectory
    create_square(wall_o, wallLength, wall_step); // To create the four walls
    create_square(rob_edge_start, rob_edge_end, robot_step) ; // To create the robot
    create_circle(C_D, c_r, x_c , y_c, charger); // To create the charger

    for (int i=0; i<pillar_no; i++)
      create_circle(P_D, c_r, x_c, y_c, i); // To create the four pillars
    fclose (fp);
    return 0;
}
```

Save the file and exit.

Step 2: create the head file below.

cd M-Drive/Documents/programs

gedit robot_function.h

You should copy the following code into the file and complete the partial code inside.

```
// The head file "robot_function.h"
#include <math.h>
#include <iostream>
#include<fstream>
using namespace std;
#define PI 3.14159265
FILE *fp; // Data file for saving the trajectory and map data
const int WB=30, dt=1, SIZE=100;
double x[5000], y[5000], RD = PI/180;
double rob_x[SIZE]={30}, rob_y[SIZE]={30}, rob_theta[SIZE]={PI/4};
// Generate the robot trajectory using robot kinematic equations
int robot_kinematics(double left_vel, double right_vel){
```

```
        double rob_vel = (right_vel+left_vel)/2;      // calculate robot speed
        double rob_turn = (right_vel-left_vel)/WB; // calculate robot turning
        rob_x[0]=30, rob_y[0]=30, rob_theta[0]=PI/4;
        for(int i=0; i<SIZE; i++){
            // put your kinematical calculation code here
            fprintf(fp, "%f, %f\n", rob_x[i], rob_y[i]);}
        return 0;
}
// Create a circle using its radius and centre coordinates for pillars and charger
void create_circle(int degrees, double r, double *x0, double *y0, int c_no){
        for (int i=0; i<360; i+=degrees){
            // put your circular creating code here
            fprintf(fp, "%f, %f\n", x[i], y[i]); }
}
// Create a square using its centre coordinates and side length for walls and robot
void create_square(double start, double end, double step){
        for(double i= start; i<end; i += step) {
            // put your wall creating code here
}}
```

Save the file and exit.

Step 3: Type the following command to compile your code.

g++ Trajectory_Map.cpp

Step 4: Type the following command to run your code.

./a.out

At this point, a new "*Trajectories-Map.txt*" file is generated in your computer.

Step 5: Then, you should display the data in "*Trajectories-Map.txt*" file using LibreOffice Calc in Ubuntu.